大语言模型原理手册

用大白话和类比，讲懂「AI 为什么能对话」

本手册面向零基础读者，不需要任何编程或数学基础。读完它，你就能理解 ChatGPT 这类大语言模型最底层的思路。

1 先从一个直觉开始

你有没有用过手机输入法的「联想词」？你打一个「复」，输入法马上提示「利」「习」「杂」等候选字；你选了「利」，它又接着提示「息」「滚」……输入法之所以猜得准，是因为它见过海量的中文文本，统计出「复」后面最常跟哪个字。

大语言模型（LLM）的本质，就是一个「超级加强版的输入法联想」。它做的只有一件事：根据前面已经出现的所有文字，预测「下一个字最可能是什么」。预测出来一个，就把这个字接上去，然后再预测下一个，如此循环，一句话、一段话就源源不断地「长」出来了。

一句话概括：语言模型 = 一台会「接龙」的机器。你给它开头，它一个字一个字地往下接。

2 第一步：把句子切成小块（分词）

计算机不能像人一样直接「读」整句话，它需要先把文字切成一小块一小块，每一小块叫一个 token（记号）。这就像做菜前先把食材切成丁，或者搭乐高前先有一盒积木。

中文怎么切？

英文单词之间有空格，切起来容易；中文没有空格，最自然的切法就是「一个字一块」。所以「复利就是利滚利」会被切成 7 个小块：复、利、就、是、利、滚、利。数字和英文则连成整体，比如「2024」和「PE」各算一块。

再加两个特殊记号

我们还会在每句话的开头和结尾，分别加一个特殊记号 <BOS>（开头）和 <EOS>（结尾）。它们就像一句话的「起跑线」和「终点线」，让模型知道一句话从哪里开始、到哪里结束。

```
tokenize("复利就是利滚利")
-> ['复', '利', '就', '是', '利', '滚', '利']
```

3　第二步：统计「谁后面跟谁」

有了切成小块的一堆句子（这叫「语料」），模型要做的第二步是：数一数——在语料里，「复」这个字后面，出现「利」几次？出现「习」几次？……出现次数越多，我们就认为它「越可能」跟在后面。

一个具体例子

假设语料里有一句「复利就是利滚利」。模型会像这样统计（看「前一个字」预测「下一个字」）：

```
复 -> 利    （出现 1 次）
利 -> 就    （出现 1 次）
就 -> 是    （出现 1 次）
是 -> 利    （出现 1 次）
利 -> 滚    （出现 1 次）
滚 -> 利    （出现 1 次）
```

如果把语料扩大成千上万句，「复」后面的「利」会一次次被数到，次数越来越多，于是模型就「学会」了：看到「复」，下一个最可能是「利」。

不只眼前一个字

只看着前一个字，太短视了。比如「利」后面可能跟「息」，也可能跟「滚」，到底跟哪个，取决于更前面的字。所以模型会多看几个字——看前两个字、前三个字……来决定下一个字。这种「看前 n-1 个字预测下一个字」的方法，就叫 n-gram（n 元语法）模型。n 越大，看得越远，但要统计的组合也越多。

> 本项目默认 n=3，也就是「看前面两个字，猜下一个字」。

4　第三步：按概率掷骰子（采样与温度）

统计完次数后，我们把「次数」换算成「概率」。比如「复」后面：利出现 100 次、习出现 20 次、杂出现 5 次，那么概率就是 利 80%、习 16%、杂 4%。

生成下一个字时，模型不是死板地永远选「利」，而是像掷一个「不均匀的骰子」：骰子各个面的面积不一样，「利」那一面最大，被掷到的机会最高，但偶尔也会掷到「习」或「杂」。这种随机性，让每次生成的句子都略有不同，更像真人在说话。

「温度」是个什么旋钮？

温度（temperature）是控制这个骰子「有多不均匀」的旋钮：

- 温度低：骰子更极端，几乎总出「利」，输出保守、死板。
- 温度高：骰子更平均，冷门字也敢出，输出大胆、多样，但也更容易乱。

你在程序里输入 `:temp 1.5`，就是把温度调高，生成更天马行空；`:temp 0.3` 则让它更循规蹈矩。

5　对照本项目：代码到底做了什么

现在回头看这个项目的代码，会发现它正好就是上面三步，几乎没有多余的东西。

训练 = 数数

```
for sentence in corpus:
    tokens = 切字(句子)
    for 每个位置 i:
        前文 = tokens[i-1], tokens[i]   # 前两个字
        下一个 = tokens[i+1]
        次数[前文][下一个] += 1          # 数一次
```

这一段就是「训练」——没有神经网络，没有 GPU，就是把语料从头到尾数一遍。

预测 = 查表 + 掷骰子

```
def 预测下一个字(前文, 温度):
    概率表 = 把 次数[前文] 换算成概率
    用温度调整概率的「均匀程度」
    按概率随机抽一个 -> 下一个字
```

生成 = 循环接龙

```
result = "复"
循环很多次:
    下一个 = 预测下一个字(result 的最后两个字)
    result += 下一个
    遇到 <EOS> 就停
输出 result
```

把这三段拼起来，就得到了一个能「说话」的模型。简单吧？这就是大语言模型的「最小可用版本」。

6　我们的小模型，和 ChatGPT 差在哪里？

思路完全一样——都是「预测下一个字」。差别在三个「量级」上。

差别一：数据量

我们的模型只读了约 100 句金融语料，像只读过一篇短文的小孩；ChatGPT 读的是互联网上几乎所有的文字，相当于把整个图书馆都读完了。读得越多，字与字的搭配规律就记得越全。

差别二：记忆方式

n-gram 是「死记硬背」：它只能记住「最近 2 个字后面跟什么」，一句话再长，它也只看眼前。而 ChatGPT 用神经网络（一种模仿大脑的数学结构）和「注意力机制」（能回头看整句话甚至整篇文章），把远距离的关联也抓住了。比如它能理解「小明把书送给小红」里的「把」和「送」隔得远，却是一对。

差别三：参数规模

我们模型的「知识」就是一张统计表，几万个数；ChatGPT 的神经网络里有上千亿个可调节的「旋钮」（叫参数），能表达极其复杂的规律。规模大了，才会涌现出理解、推理、甚至「创造力」。

> 打个比方：n-gram 像是「背熟了一本词语搭配词典」，而 GPT 像是「读遍天下书、还懂得举一反三的学霸」。但两者追根究底，都在做同一件事——预测下一个字。

7 为什么「预测下一个字」能产生智能？

你可能会怀疑：就预测一个字，凭什么叫「智能」？关键在于——要把下一个字预测得准，模型被迫要学会理解。

- 要接对「1+1=__」，它得会算术；
- 要接对「北京是中国的__」，它得懂常识；
- 要接对「这篇文章的中心思想是__」，它得会阅读和理解；
- 要接对「推荐一个适合定投的__」，它得懂金融知识。

所以，「预测下一个字」这个看似简单的任务，其实是一个「逼着模型变聪明」的终极考试。模型为了考高分，不得不把语言、知识、逻辑都内化进去。这就是大语言模型如此强大的根本原因。

8 AI 在金融领域能做什么？

把「预测下一个字」的能力放到金融方向，就有了很多真实应用，我们这个项目只是其中最小的一角。

- 智能客服/投教助手：像本项目一样，回答股票、基金、理财等常见问题。
- 研报与资讯生成：根据数据自动撰写财报解读、市场点评。
- 舆情与情绪分析：从新闻、社交文本里判断市场情绪是乐观还是悲观。
- 风险提示：结合规则，提示投资者注意杠杆、止损、仓位等风险。
- 知识抽取：从海量公告、合同里自动提取关键信息。

> 重要提醒：语言模型会「一本正经地编」，在金融这种高风险场景，AI 只能当辅助，不能代替人做投资决策。

9 总结：一张图看懂

整个大语言模型的逻辑，可以用一条流水线概括：

```
大量文字(语料)
    |  切字
    ▼
一串 token
    |  统计「谁后面跟谁」
    ▼
概率表（或神经网络）
    |  掷骰子选下一个字
    ▼
一个字一个字接下去 -> 一段话
```

记住这五个关键词，你就入门了：

- token：被切碎的文字小块；
- 语料：给模型「读」的大量文字；
- 训练：统计规律、调整参数的过程；
- 采样：按概率随机选下一个字；
- 温度：控制随机程度的旋钮。

10　给你留一个亲手实验

读完原理，建议回到程序里做这几个实验，把「纸上原理」变成「亲眼所见」：

- 实验 1：把 `:temp` 从 0.2 调到 1.8，感受输出从「死板」到「乱来」的变化——这就是温度。
- 实验 2：往 `finance_corpus.txt` 里加 10 句金融句子，重启后再 `:gen`，看输出是否更像样——这就是数据的力量。
- 实验 3：把 `ngram_model.py` 里的 `n` 改小（2）再改大（4），观察变化——这就是「看多远」的权衡。

做完这三个实验，你不仅听懂了原理，还亲手验证了原理。祝你学习顺利！